Sistemas Operativos (Caso de Aplicación)

Clase 1

Universidad Tecnológica Nacional - FRH

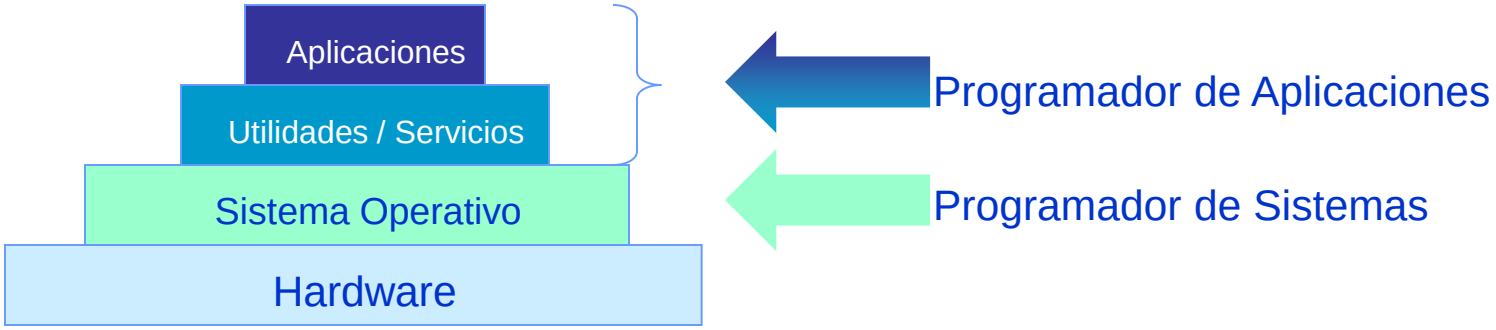
Técnicas Digitales III

Ciclo Lectivo 2020

Autor: Mariano Mercado

Introducción

- Sistema Operativo ¿Que es?
- Es un programa de control que se ocupa de:
  - Administrar los recursos de la computadora.
  - Administrar la ejecución de los diferentes programas en muchos casos pueden ser de diferentes usuarios.
  - Facilitar la tarea del programador permitiéndole acceso a los recursos de manera independiente del hardware.
  - Proveer servicios a los programas de aplicación a través de un conjunto de llamadas Standard.
- Estas acciones se resuelven a través de una implementación que puede representarse en capas:



Introducción

Ministerio de Educación, Cultura, Ciencia y Tecnología
Presidencia de la Nación

Resolución Ministerial 1254/2018

Actividades Reservadas al título de Ingeniero Electrónico:

1. **Diseñar, proyectar y calcular** sistemas, equipos y dispositivos de generación, transmisión, y/o procesamiento de campos y señales, analógicos y digitales; circuitos integrados; **hardware de sistemas de cómputo de propósito general y/o específico y el software a él asociado**; hardware y **software de sistemas embebidos y dispositivos lógicos programables**; sistemas de automatización y control; sistemas de procesamiento y de comunicación de datos y sistemas irradiantes.

2. Proyectar, dirigir y controlar la construcción, implementación, mantenimiento y operación de lo mencionado anteriormente.

3. Validar y certificar el funcionamiento, condición de uso o estado de los sistemas mencionados anteriormente.

4. Proyectar y dirigir lo referido a la higiene y seguridad en su actividad profesional.

Clasificación de los Sistemas Operativos

Monotarea – Monousuario

- Transfiere el control de la máquina a la aplicación que va a ejecutarse, y solo interviene a demanda de ésta mediante alguna llamada a los servicios de su kernel, o cuando la aplicación finaliza y devuelve el control.

Multitarea – Monousuario

- Interfaz para un solo usuario, pero pueden mantener en memoria múltiples aplicaciones en forma estable y dentro de un entorno de protección

Multiusuario

- Estos sistemas son los mas poderosos y normalmente los mas eficientes: MVS, para los mainframes, UNIX (o cualquiera de sus versiones free como LINUX, o free BSD, por ejemplo) son los mejores exponentes de este tipo de sistemas. Microsoft tiene versiones denominadas “Server” que soporta multiusuarios

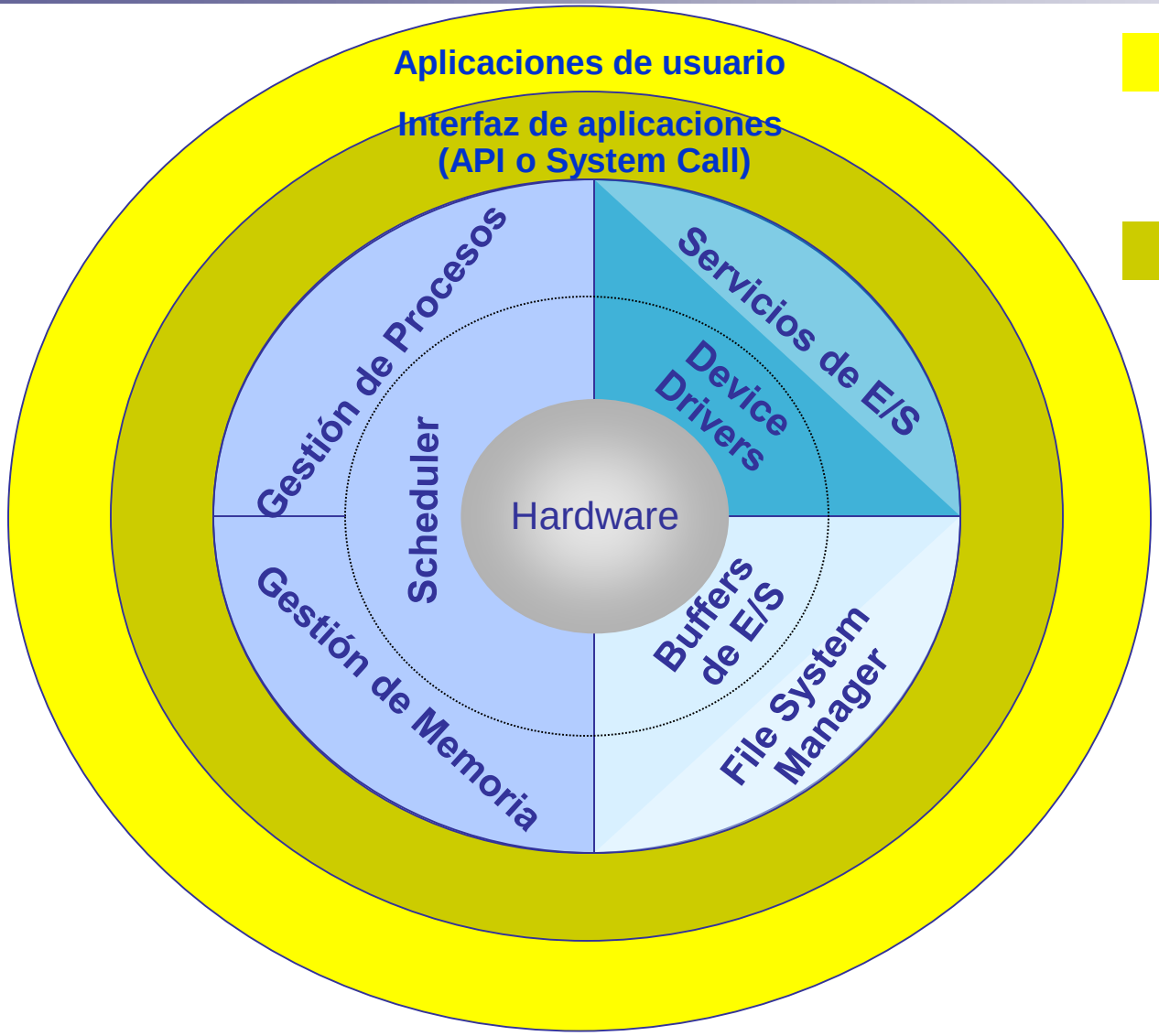
Funciones de un Sistema Operativo

- Gestión del procesador
  - Gestión del tiempo de procesamiento para cada tarea (scheduling de procesos).
- Gestión de la Memoria.
  - Asignación de memoria RAM para las aplicaciones aplicando criterios basados en la MMU del procesador.
  - Gestión de la Memoria Virtual
  - Gestión de la memoria cache
- Gestión de los dispositivos de E/S.
  - Acceso al hardware de manera transparente para las aplicaciones.
  - Manejo de la concurrencia de acceso en sistemas multiusuario o multitarea especialmente

Funciones de un Sistema Operativo

- Gestión del storage – File Systems
  - (File Systems en los medios masivos de almacenamiento: discos rígidos, CD-ROMs, DVDs).
- Interfaz para las Aplicaciones.
  - Funciones desarrolladas por el Sistema Operativo, disponibles para las aplicaciones. Se las conoce como System Calls (Call Gates !!).
  - En los multitarea se manejan mediante este subsistema, los diferentes niveles de privilegio que posea el Sistema Operativo (y que dependen del procesador utilizado en el sistema)
- Interfaz para los usuarios.
  - Manejo de interfaces sencillas para usuarios no expertos
    - GUI
    - Texto
    - Combinación de ambas

Modelo derivado de las funciones



Programas de Aplicación y Utilitarios Ejecutan en el menor nivel de privilegio

Capa de interfaz para acceso a los servicios del S.O. Por parte de las aplicaciones

KERNEL

Hardware
(lo que golpeamos a causa de los estándares de calidad de algunos S.O.'s ....)

Linux: Introducción

- Sistema Operativo Unix-like. Basado en POSIX (Portable OS based on UNIX)
- Kernel monolítico (programa único + módulos)
  - Opuesto a los Sistemas MicroKernel que reinaron en los 70's.
- Diseñado bajo el concepto Lightweight Processes (LWP)
- Soporta varios File Systems (IBM AIX, SGI Irix, FAT32, etc)
- Puede ser muy pequeño. Se puede ajustar a un floppy 3"1/2, o USB.
- Es libre. Podemos instalarlo y modificar su código sin otra limitación que nuestro hardware.
- Versiones. Se representan actualmente con cuatro números separados por puntos (A.B.C.D). Por ejemplo: **2.6.34.14**, o **5.2.8**.
  - El A es la Versión, solo se cambia cuando se produce un gran cambio en el núcleo.
  - El B indicaba hasta el kernel 2.6 si es un kernel estable (par) o en desarrollo (impar), actualmente se fijó a este número como el sub-version del kernel.
  - El C indica revisiones mayores.
  - El D indica correcciones de bugs, parches de seguridad, etc.

El Kernel de Linux

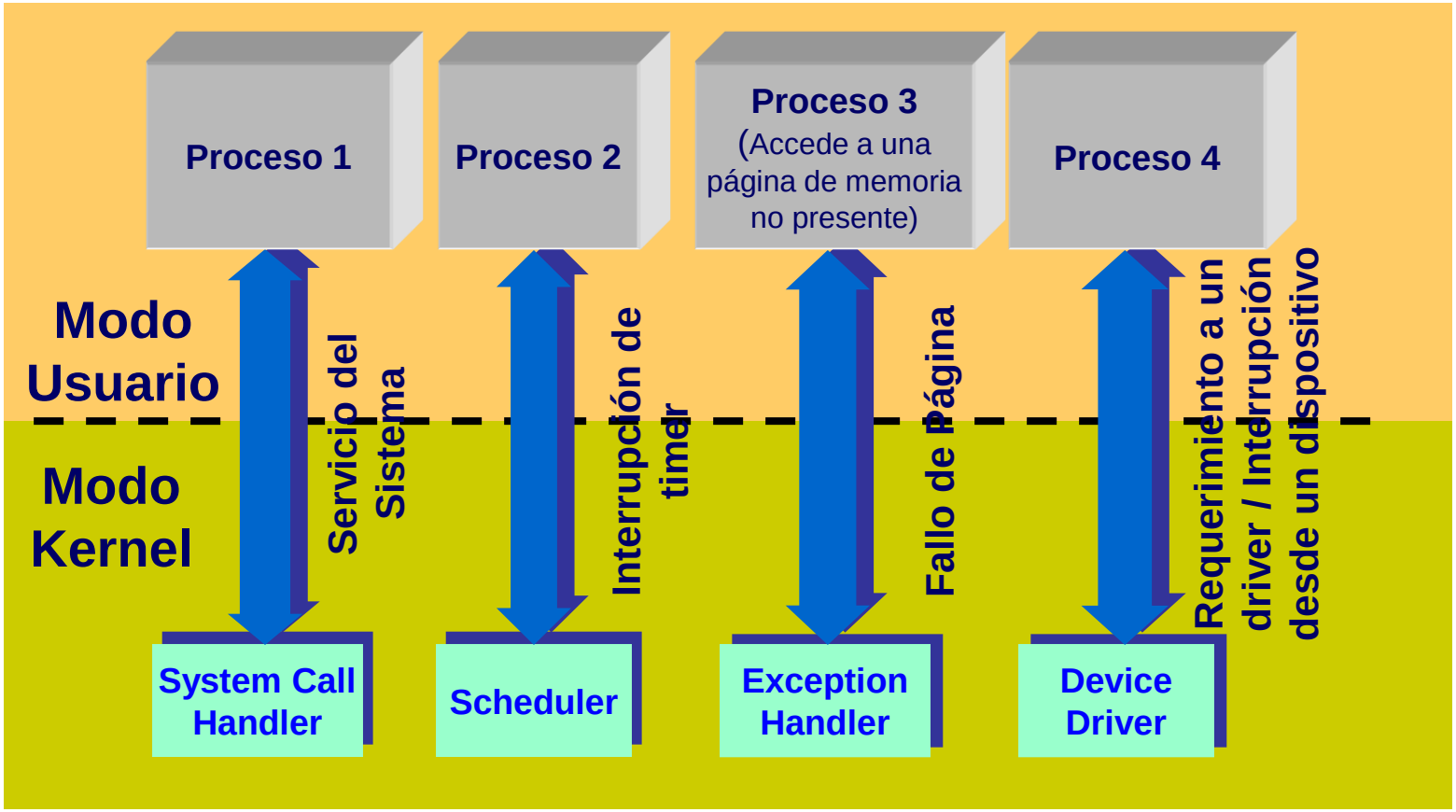
- Es el principal programa del SO.
- Soportado por las diferentes arquitecturas:
  - ARM (Palm, Nokia, Nintendo DS, Freescale)
  - Alpha (Compaq, Samsung)
  - Atmel
  - Intel (IA32 e Intel64)
  - Familia 68K (Motorola)
- Aprovecha las capacidades del hardware
  - Maneja el acceso a los recursos hardware específicos. A través de Device Drivers.
  - Provee servicios de acceso al hardware a los programas de usuario.
- Es reentrante
  - Múltiples procesos acceden al kernel de manera simultánea.



Linus Torvalds

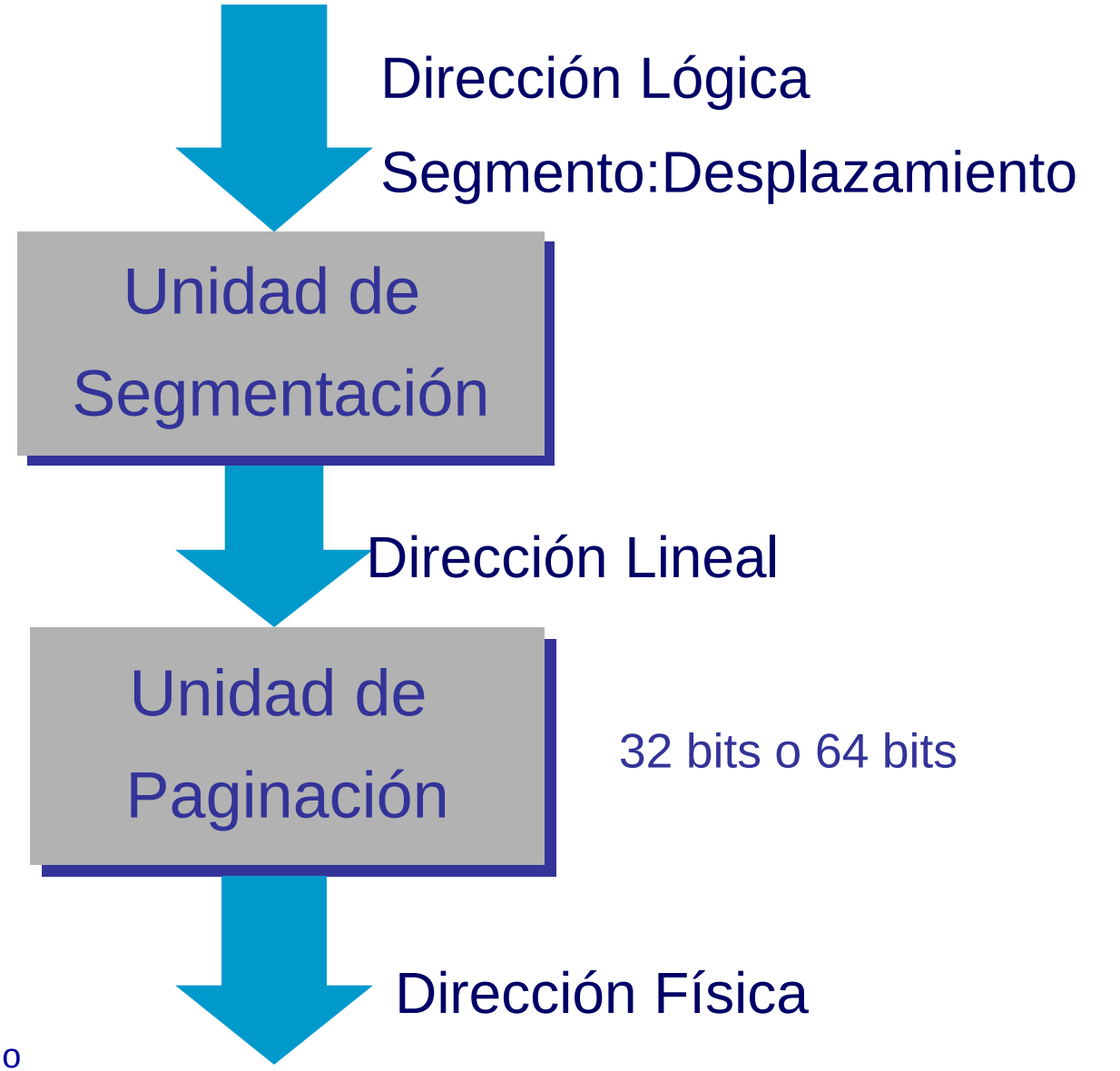
El Kernel

- Gestiona los niveles de Protección
  - Ejecución en modo Kernel.
  - Ejecución en modo Usuario.
  - Maneja Stacks separados.



Manejo de Memoria

Modelo:



Manejo de Memoria: Tabla GDT

Linux's GDT	Segment Selectors
null	0x0
reserved	
reserved	
reserved	
not used	
not used	
TLS #1	0x33
TLS #2	0x3b
TLS #3	0x43
reserved	
reserved	
reserved	
kernel code	0x60 (__KERNEL_CS)
kernel data	0x68 (__KERNEL_DS)
user code	0x73 (__USER_CS)
user data	0x7b (__USER_DS)

Linux's GDT	Segment Selectors
TSS	0x80
LDT	0x88
PNPBIOS 32-bit code	0x90
PNPBIOS 16-bit code	0x98
PNPBIOS 16-bit data	0xa0
PNPBIOS 16-bit data	0xa8
PNPBIOS 16-bit data	0xb0
APMBIOS 32-bit code	0xb8
APMBIOS 16-bit code	0xc0
APMBIOS data	0xc8
not used	
not used	
not used	
not used	
not used	
double fault TSS	0xf8

Manejo de Memoria: Tabla GDT

Modelo FLAT

- Descriptores en GDT
  - Kernel Code Segment (KCS)
  - Kernel Data Segment (KDS)
  - User Code Segment (UCS)
  - User Data Segment (UDS)

Segment	Base	G	Limit	S	Type	DPL	D/B	P
user code	0x00000000	1	0xfffff	1	10	3	1	1
user data	0x00000000	1	0xfffff	1	2	3	1	1
kernel code	0x00000000	1	0xfffff	1	10	0	1	1
kernel data	0x00000000	1	0xfffff	1	2	0	1	1

Modelo Kernel / Proceso

- Proceso: Instancia de un programa en ejecución, o un contexto de ejecución.
  - Tienen un espacio de direccionamiento determinado (direcciones de memoria que tienen permitidas para acceder)
  - Normalmente trabajan en modo User.
  - Cuando requieren servicios del kernel, switchean a modo kernel. Una vez resuelto el requerimiento regresan a modo User
- El kernel hace que cada proceso “vea” una CPU dedicada. Es un administrador de procesos
- Existe un grupo de procesos privilegiado llamados “kernel threads”.
  - Corren en modo Kernel en el espacio de direccionamiento del kernel.
  - No interactúan con el usuario.
  - No tienen terminal asociada.
  - Se crean durante el startup y mueren en el shutdown

Procesos

- El kernel debe saber sobre cada proceso:
  - Su prioridad.
  - Su estado.
  - Su espacio de direcciones asignado.
  - Sus archivos abiertos y en acceso.

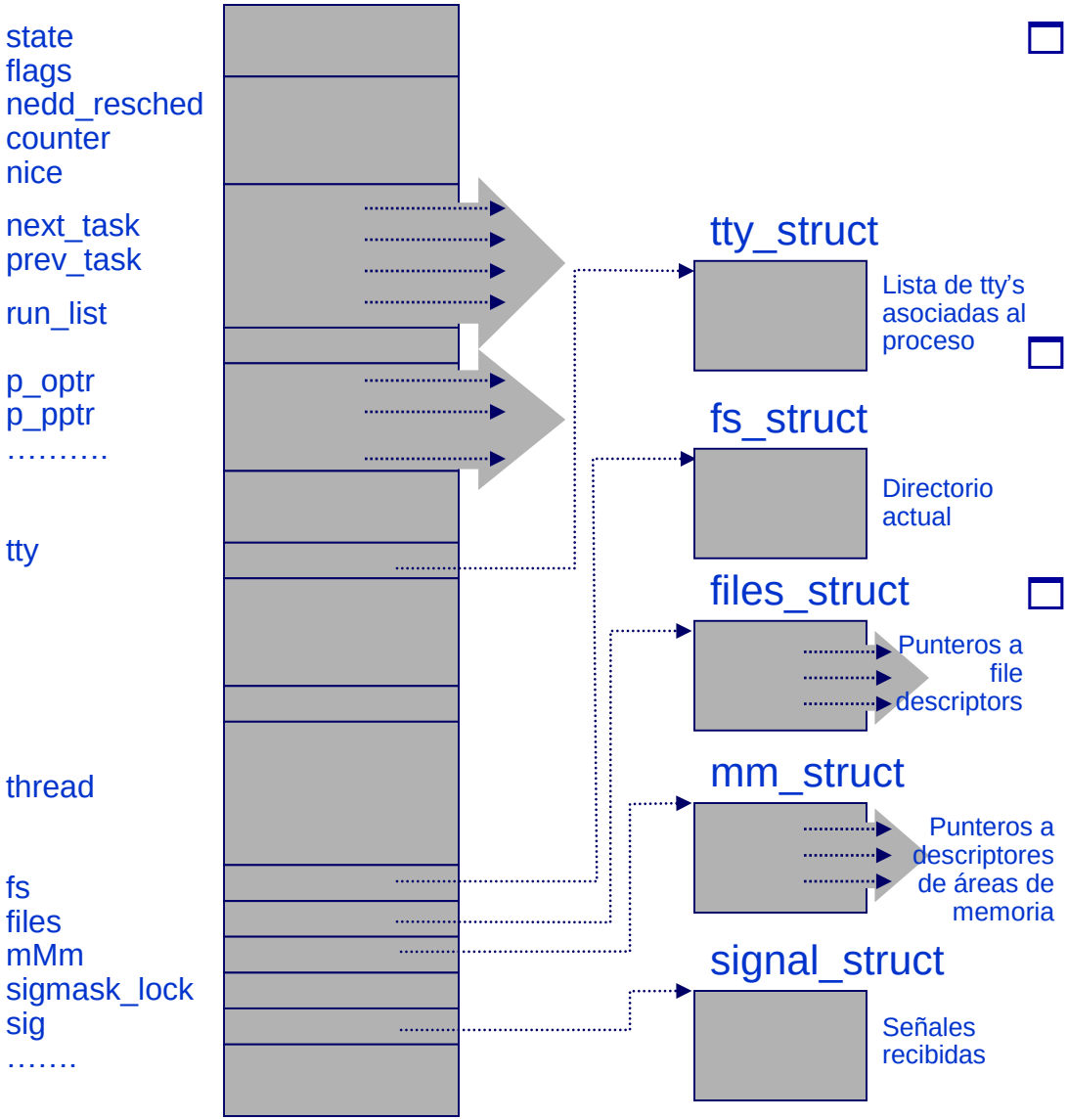
Donde guarda toda esta información el kernel?

VER

- Descriptores de Proceso, estructura task_struct,

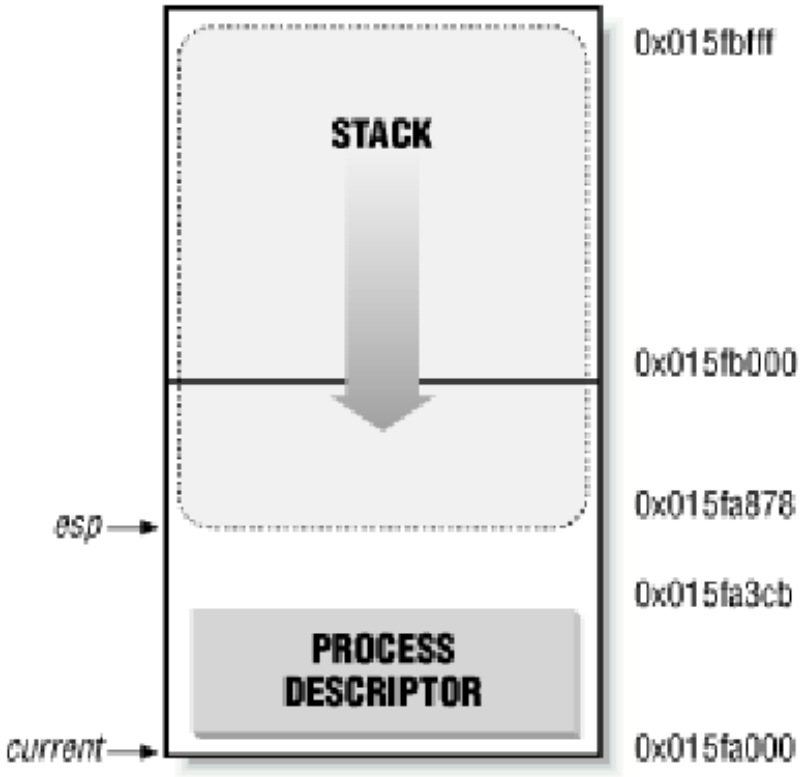
  El kernel define un array de estas estructuras denominado task_array.

Descriptor de Procesos



- Tiene por objeto darle al Sistema Operativo la visión clara de que está haciendo el proceso.
- Definido en la declaración de ***task_struct*** **VER**
- Dada la complejidad de esta estructura algunos campos son referencias a otras estructuras como la tabla de file descriptors del proceso

Descriptor de Proceso: Memory Allocation




```
union task_union
{
    struct task_struct task;
    unsigned long stack[2048];
};
```

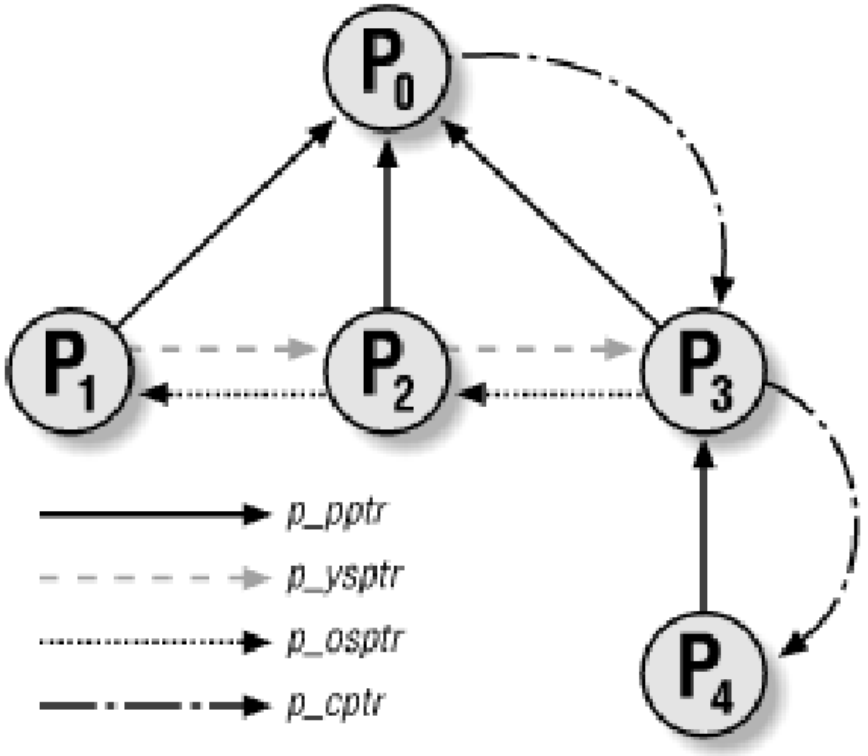
- Los procesos usan un Stack del Segmento de Datos del Kernel.
- Cada proceso usa 8Kb de memoria en Modo Kernel para el Descriptor + Stack.
- Los stacks expanden hacia la base de memoria (el registro ***esp*** se decrementa)
- Macro current: Devuelve un puntero al Process Descriptor.

Identificando un proceso

- Cada proceso tiene un Descriptor de Proceso que lo define unívocamente.
- Los punteros a los descriptores de proceso sirven para identificarlos. Son Números de 32 bits.
- Los UNIX identifican a los procesos con un Process ID (PID). Los PID van desde 0 a 32767.
- El Process ID corresponde al campo ***pid*** en el descriptor de proceso (***task_struct***)

Parentesco entre procesos

- En el descriptor de procesos se dispone de los siguientes campos para reflejar parentesco
  - p_pptr (parent)
  - p_cptr (child)
  - p_ysptr (younger sibling)
  - p_osptr (older sibling)

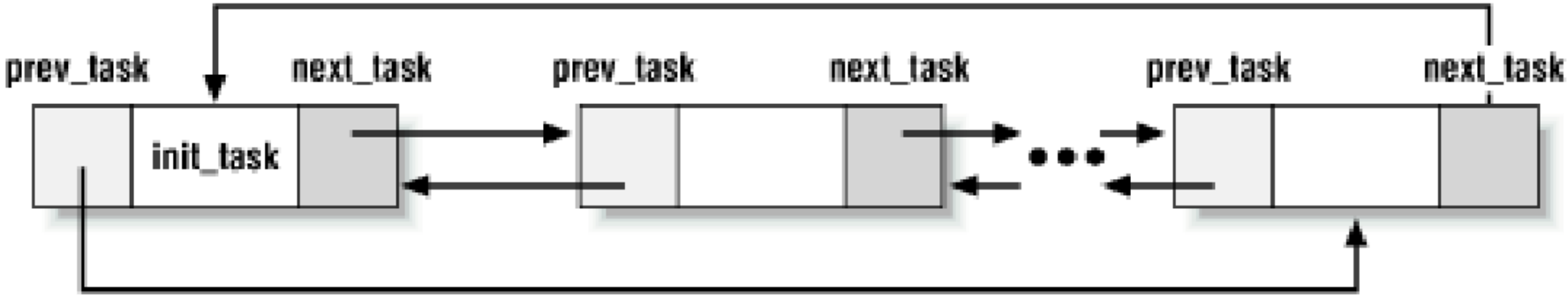


Estados de un proceso

- Corresponden al campo ***state*** de task_struct. Valores posibles (flags)
  - TASK_RUNNING: El proceso requiere ser ejecutado.
  - TASK_Interruptible: El proceso esta durmiendo a la espera de un retorno de la system call a la cual llamo. Si recibe dicha respuesta u otra señal, interrupción, etc, se despierta y pasa a estado TASK_RUNNING.
  - TASK_Uninterruptible: El proceso esta durmiendo a la espera de un retorno de la system call a la cual llamo. Aun cuando reciba una señal, o interrupción, seguirá en este estado y no pasa a TASK_RUNNING. La system call a la cual llamo, cuanto tenga el dato para retornarle la volverá al estado RUNNING.
  - TASK_Stopped: La ejecución del proceso ha sido detenida (En pausa). Puede luego ser puesta en RUNNING nuevamente.
  - TASK_ZOMBIE: Un proceso queda en este estado, cuando el proceso termina, pero el proceso que lo creó (su padre) no “reconoce” recibiendo la información de salida de su hijo.

Lista de Procesos

- ***Task array***: Lista doblemente enlazada de punteros a estructuras de tipo ***task_struct***
- La cantidad de elementos se guarda en la variable ***N_TASKS.*** El campo ***pid*** es el índice de este arreglo.
- Los campos ***prev_task*** y ***next task*** de los descriptores de procesos (***task_struct***), vinculan los elementos.



Implementación de listas doblemente enlazadas en Linux

- Linux utiliza numerosas listas doblemente enlazadas.
- Buscando un estilo de programación homogéneo se define la siguiente estructura genérica:

```
struct list_head {
        struct list_head *next, *prev;
        };
```

- La misma se encuentra en el archivo */usr/src/linux-[version]/includes/linux/list.h*



Runqueue

- Por una cuestión de eficiencia, el kernel mantiene una lista separada de los procesos que están en estado TASK_RUNNING.
- La variable ***nr_running*** mantiene la cantidad de procesos en este estado.
- La lista está encabezada por ***init_task*** (descriptor de proceso del proceso 0 o swapper)
- El campo ***run_list*** de ***task_struct*** es una estructura ***list_head*** que permite “linkear” la tarea dentro de la lista runqueue.
- Funciones que utiliza el kernel para administrar la runqueue:
  - ***add_to_runqueue()***: Inserta un proceso a la lista
  - ***del_from_runqueue()***: remueve un proceso de la lista
  - ***move_first_runqueue()***: Mueve un proceso al principio de la lista
  - ***move_last_runqueue()***: Mueve un proceso al final de la lista

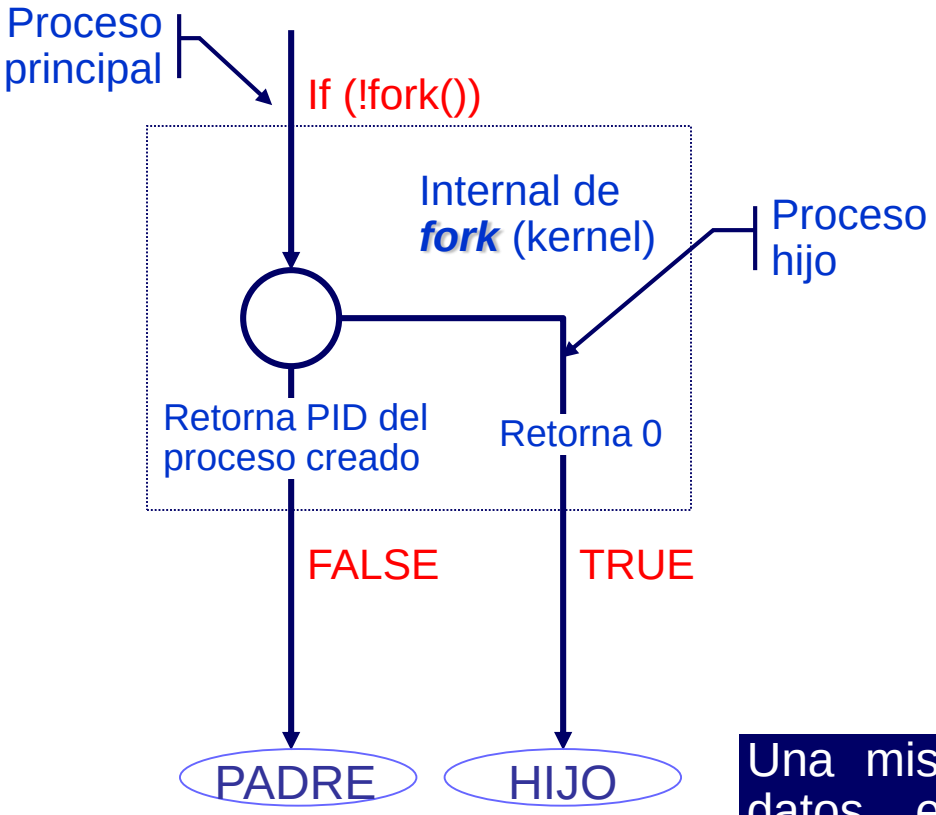
Creación de procesos

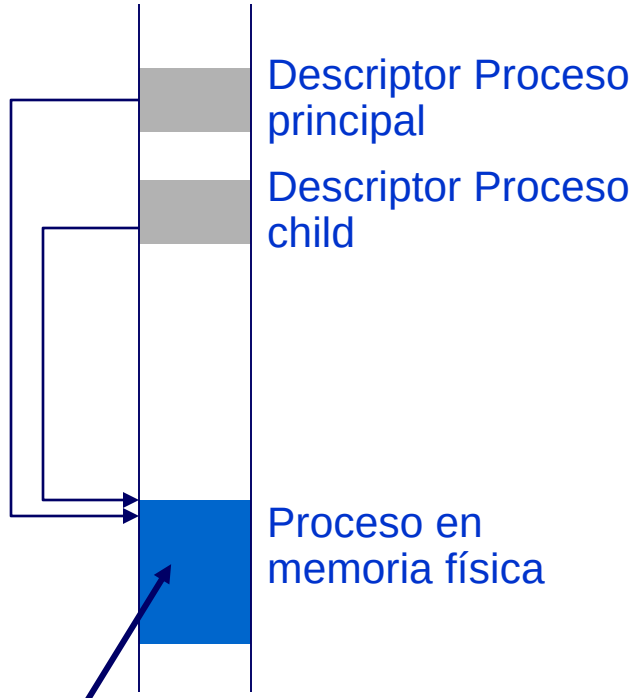
Lightweight processes.

- Cada vez que un proceso crea a otro, se crea un nuevo Descriptor de Proceso, y se transfiere sus recursos. **Pero no se duplican los recursos (Código ni datos).**
- Copy on write. El kernel detecta el acceso a modificar un recurso por parte del child y en ese momento le crea una copia propia.
- Comparten las estructuras del kernel, y el espacio de direcciones en modo User.

Creación de procesos: fork ()

- Se emplean diferentes system calls para crear procesos. La mas utilizada es ***fork ()***.





Una misma copia física del código y los datos en memoria apuntada por dos descriptores de proceso diferentes. Una copia = Dos procesos. (Lightweight Process)

Scheduling de procesos

- Linux divide el tiempo de la CPU en períodos (o épocas)
- En cada época de la CPU, Linux (como UNIX) asigna un quantum de tiempo a cada proceso -> time slicing.
- La duración del quantum es critica en la performance
- Linux programa al timer tick de modo de generar una interrupción (tick) cada 10 mseg. aproximadamente.

Conmutación de Procesos

- Es responsabilidad del scheduler (sched.c) e incluye:
  - Contexto de Hardware
    - Directorio de Páginas para cambiar al nuevo espacio de direcciones
    - Stacks de Modo User y Modo Kernel, mas los Registros propios de los procesadores IA-32
  - Registros adicionales que al igual que en el procesador de Intel hay que manejarlo en forma manual, por falta de soporte en el hardware
    - Registros de FPU
    - Registros de control y Debug
  - Utiliza una macro llamada ***switch_to()*** en lugar de utilizar un jump far al descriptor del TSS
    - void switch_to (struct task_struct *prev, struct task_struct *next, struct task_struct *last)

Conmutación de Procesos

- Linux mantiene una única TSS por cada CPU. **VER**
- A pesar del no uso de jmp far para conmutar no puede prescindir de al menos un TSS por cada CPU:
  - Al aumentar el nivel de privilegio de una tarea, el procesador busca en el TSS actual (cuyo selector contiene el registro TR), los valores de SS y ESP correspondientes al mayor nivel de privilegio.
  - Para acceder a E/S desde una tarea cuyo CPL no sea el adecuado debe consultarse el IO BitMap del TSS de la tarea.
- Al usar un único TSS por CPU, Linux debe actualizarle determinados campos en cada process switch.

Conmutación de Procesos

- El contexto de ejecución de cada tarea se almacena en una estructura del tipo ***thread*** en ***task_struct***. VER
- Esta estructura puede ser tomada por cualquier CPU presente en el sistema ya que contiene los datos que de guardarse en la TSS no podrían ser tomados por otra CPU.
- Observaciones:
  - Solo guarda los registros de propósito general estrictamente necesarios. El resto va al stack.
  - Se almacena el estado de la FPU, y debug registers que no son contemplados en el TSS
  - Se mantiene el bitmap de E/S del proceso en esta estructura

Interrupciones y Excepciones

- Interrupciones
  - Sincrónicas: Excepciones (se producen una vez completada la instrucción en curso).
  - Asincrónicas: Interrupciones de Hardware.
- Tipo de Interupción en Linux:
  - 0-31 excepciones y NMI
  - 32-47 IRQ's
  - El resto para Int's soft, Linux usa la 128 (0x80) (system calls, cuando un proceso user la ejecuta, se pasa a modo kernel).
- Interrupt Handling ≠ Process Switching

Manejo de las Interrupciones en Linux

Rango	Uso
0-19 (0x0-0x13)	Nonmaskable interrupts and exceptions
20-31 (0x14-0x1f)	Reservado por Intel
32-127 (0x20-0x7f)	External interrupts (IRQs)
128 (0x80)	Programmed exception for system calls
129-238 (0x81-0xee)	External interrupts (IRQs)
239 (0xef)	Local APIC timer interrupt
240-250 (0xf0-0xfa)	Reserved by Linux for future use
251-255 (0xfb-0xff)	Interprocessor interrupts

Vectores de Interrupción en Linux

IDT

- Asocia interrupciones y excepciones con sus respectivos handlers.
- Es inicializada luego del POST, de acuerdo a lo establecido en la diapositiva anterior
- Se inicializa antes que el kernel habilite las interrupciones.
- Puede contener tres tipos de descriptores:
  - Task Gate (Linux NO los usa!!!)
  - Interrupt Gate (para atender interrupciones)
  - Trap Gate (para atender excepciones)
- Linux diferencia de intel los nombres...

IDT
Terminología Linux

- Interrupt Gate:
  - Solo para modo Kernel
  - Son las interrupt gates de intel con DPL=0.
- System Gate: es la trap gate de Intel.
  - 3 (int3)
  - 4 (into)
  - 5 (bound)
  - 128 (system call) (equivalente a int 21 en DOS). Funciona como una interrupción.
  - Se ejecutan en modo user.
- Trap Gate: es una trap gate Intel pero solo accesible en modo kernel.

Handlers de Excepciones del procesador

#	Exception	Exception handler	Signal
0	Divide error	divide_error( )	SIGFPE
1	Debug	debug( )	SIGTRAP
2	NMI	nmi( )	**None**
3	Breakpoint	int3( )	SIGTRAP
4	Overflow	overflow( )	SIGSEGV
5	Bounds check	bounds( )	SIGSEGV
6	Invalid opcode	invalid_op( )	SIGILL
7	Device not available	device_not_available( )	SIGSEGV
8	Double fault	double_fault( )	SIGSEGV
9	Coprocessor segment overrun	coprocessor_segment_overrun( )	SIGFPE
10	Invalid TSS	invalid_tss( )	SIGSEGV
11	Segment not present	segment_not_present( )	SIGBUS
12	Stack exception	stack_segment( )	SIGBUS
13	General protection	general_protection( )	SIGSEGV
14	Page Fault	page_fault( )	SIGSEGV
15	Intel reserved	None	None
16	Floating-point error	coprocessor_error( )	SIGFPE
17	Alignment check	alignment_check( )	SIGBUS
18	Machine check	machine_check( )	None
19	SIMD floating point	simd_coprocessor_error( )	SIGFPE

Manejo de Interrupciones

- El proceso interrumpido está TASK_RUNNING.
- Tiene asignado un quantum para su ejecución.
- La Interrupción no debe superar ese quantum
  - **Acciones críticas.** Se resuelven en el handler con las interrupciones deshabilitadas
  - **Acciones No críticas.** Se resuelven en el handler con las interrupciones habilitadas
  - **Acciones No críticas diferibles.** Se ejecutan en algún momento oportuno fuera del handler de interrupción. (Bottom half)

Temporizaciones

- El kernel lleva a cabo dos tipos de temporizaciones:
  - Mantiene la fecha y hora que retorna a los programas de aplicación como respuesta a las System Calls time(), ftime(), y gettimeofday().
  - Mantiene Timers, es decir, mecanismos que le notifique a él o a las aplicaciones que ha transcurrido un lapso de tiempo predeterminado

Temporizaciones

- Hay tres relojes de hardware en una PC:
  - El RTC (Motorola 146818).
    - Genera IRQ8, para interrumpir desde 2 hasta 8192 Hz.
    - Se accede por /sbin/clock
    - Actúa sobre el dispositivo /dev/rtc
    - Se accede a través de las direcciones 70h y 71h de E/S
  - Time Stamp Counter
    - Registro homónimo (TSC) de 64 bits interno al procesador a partir del Pentium. Cuenta a la velocidad del clock
  - Programable Interval Timer (8254)
    - Genera la IRQ0 (Timer Tick)
    - Es programado en el startup para interrumpir a 100 hz

Timer Interrupt Handler

- Actualiza el tiempo transcurrido desde el startup
- Actualiza Hora y Fecha
- Determina cuanto hace que corre el proceso actual y eventualmente decide si lo “schedula”.
- Actualiza los time stamps de uso de los recursos del sistema
- Chequea los estados de los timers de software en uso por parte de las aplicaciones

IPC (Inter Process Communication)

- Como hacer que dos o más procesos intercambien información? → IPC
- Primero debemos contar con más de un proceso.

```
fork()     Devuelve 0 al HIJO
           Devuelve PID del hijo al PADRE

execlp ("pepe.x", arg0, arg1,…, argn);
           Reemplaza el código del hijo
           con el del programa pepe.x
           Se puede a su vez enviar argumentos
           al programa
```

IPC (Inter Process Communication)

```
main()
    {
        pid_t pid;
        int rv;
        switch(pid=fork()) {
            case -1:
                perror("fork");  /* something went wrong */
                exit(1);         /* parent exits */
            case 0:
                printf(" CHILD: This is the child process!\n");
                printf(" CHILD: My PID is %d\n", getpid());
                printf(" CHILD: My parent's PID is %d\n", getppid());
                exit();
            default:
                printf("PARENT: This is the parent process!\n");
                printf("PARENT: My PID is %d\n", getpid());
                printf("PARENT: My child's PID is %d\n", pid);

        }
    }
```

IPC: Señales

- Los procesos pueden señalizarse entre sí o puede hacerlo el kernel.
- Las señales es un mensaje muy corto que es enviado a un proceso, donde no cabe lugar para argumentos o información adicional. El único dato que llega es el tipo o numero de señal.
- Las señales tienen 2 propósitos: a. Avisar a otro proceso sobre algún evento ocurrido, o b. Forzar a otro proceso a ejecutar el handler de atención a la señal.
- Salvo algunas excepciones, las señales pueden ignorarse, interceptarse para cambiar el handler o ejecutar su handler por default.

IPC: Señales

#	Nombre	Acción Default	Descripción	POSIX
1	SIGHUP	Abort	Se desconectó la terminal asociada al proceso	Si
2	SIGINT	Abort	Interrupción desde el teclado (CTRL+C)	Si
3	SIGQUIT	Dump	Quit desde el teclado (CTRL+\)	Si
4	SIGILL	Dump	Instrucción ilegal	Si
5	SIGTRAP	Dump	Breakpoint para debugging	No
6	SIGABRT	Dump	Abnormal termination	Si
6	SIGIOT	Dump	Equivalente a SIGABRT	No
7	SIGBUS	Abort	Bus error	No
8	SIGFPE	Dump	Floating-point exception	Si
9	SIGKILL *	Abort	Fuerza la terminación del proceso.	Si
10	SIGUSR1	Abort	Dispponible para el proceso	Si
11	SIGSEGV	Dump	Referencia inválida a memoria	Si
12	SIGUSR2	Abort	Dispponible para el proceso	Si
13	SIGPIPE	Abort	Escritura en un pipe sin procesos lectores	Si
14	SIGALRM	Abort	Real-timer clock	Si
15	SIGTERM	Abort	Terminación de un Proceso	Si
16	SIGSTKFLT	Abort	Coprocessor stack error	No

IPC: Señales

#	Nombre	Acción Default	Descripción	POSIX
17	SIGCHLD	Ignore	Proceso Child STOPPED o terminó	Si
18	SIGCONT	Continue	Reasume la ejecución, si estaba STOPPED	Si
19	SIGSTOP *	Stop	Detiene al proceso (lo pone STOPPED)	Si
20	SIGTSTP	Stop	Idem SIGSTOP enviada por la tty (CTRL+Z)	Si
21	SIGTTIN	Stop	Proceso Background requiere entrada	Si
22	SIGTTOU	Stop	Proceso Background requiere salida	Si
23	SIGURG	Ignore	Condición **Urgent** en un socket	No
24	SIGXCPU	Abort	Límite de tiempo de CPU excedido	No
25	SIGXFSZ	Abort	Tamaño límite de Archivo excedido	No
26	SIGVTALRM	Abort	Virtual timer clock	No
27	SIGPROF	Abort	Profile timer clock	No
28	SIGWINCH	Ignore	Window resizing	No
29	SIGIO	Abort	Ahora es posible una Operación de I/O	No
29	SIGPOLL	Abort	Equivalente a SIGIO	No
30	SIGPWR	Abort	Power supply failure	No
31	SIGSYS	Abort	system call errónea	No
31	SIGUNUSED	Abort	Equivalente a SIGSYS	No

IPC: Señales

1. Por línea de comando:

kill –s <Nombre_Señal> PID

2. A través de una función

```
int kill (pid_t pid, int sig);
```

- Si pid es positivo, entonces la señal sig es enviada a pid. En este caso, se devuelve 0 si hay éxito, o un valor negativo si hay error.

- Si pid es 0, entonces sig se envía a cada proceso en el grupo de procesos del proceso actual.

- Si pid es igual a -1, entonces se envía sig a cada proceso, excepto al primero (pid=0), desde los números más altos en la tabla de procesos, hasta los más bajos.

- Si pid es menor que -1, entonces se envía sig a cada proceso en el grupo de procesos -pid.

* VALOR DEVUELTO Si hay éxito, se devuelve cero. Si hay error, se devuelve -1, y se actualiza errno apropiadamente.

Señales, Handlers

Cuando un proceso recibe una señal, éste puede realizar alguna de las siguientes acciones:

a. Ignorar la señal (si esta se encuentran enmascarada)

b. Atender la señal con un handler default

c. Atender la señal con un handler propio (cuando la señal es trapeada utilizando la system call signal())

Según el tipo de señal, la acción default podra ser:

- Abort: El proceso es destruido, finaliza (killed)
- Dump: El proceso es detruido y se genera un archivo de debug.
- Ignore: Se ignora la señal.
- Stop: El proceso detiene su ejecución.
  (Pasa a TASK_STOPPED state)
- Continue: El proceso retoma su ejecución
  (Pasa a TASK_RUNNING state)

Señales, System Calls

Las siguientes son las system calls asociadas al manejo de señales de mayor utilización:

- kill (): Envia una señal a un proceso.

- signal (): Cambia la acción asociada a una señal. (trapea la señal asignándole un nuevo handler).

- sigprocmask(): Modifica el set de señales enmascaradas (el proceso puede filtrar algunas señales que no desea utilizar).

Otros IPC

- Pipes and Named Pipes (FIFOS)
  - Ideales para la comunicacion entre procesos tipo Cliente / Servidor. Un proceso envia datos al pipe y otro proceso extrae los datos del pipe.
- Semaphores (semáforos)
  - Ideales para recursos compartidos con acceso limitado. Por ejemplo cuando dos procesos requieren escribir en un mismo bloque de memoria.
- Message queues
  - Ideales para la comunicacion asincrónica entre procesos. Como las señales pero con posibilidad de enviar información.
- Shared Memory
  - Ideales para interacciones de gran cantidad de datos entre procesos.
- Sockets
  - Permiten comunicar procesos via una red de datos.

IPC, Pipes

- Flujo de información en un solo sentido (one-way). Un proceso inserta datos en el pipe y otro los extrae.
- Para crear un pipe se utiliza la system call *pipe().* Esta system call devuelve un par de file descriptors. Uno para escritura y uno para lectura. Por ejemplo:

```
int pfds[2];
pipe(pfds);
```

- Para leer en el pipe se utiliza read() sobre el 1er fd.

```
char buf[30];
read(pfds[0], buf, 5);
```

- Para escribir en el pipe se utiliza write() sobre el 2do fd.

```
write(pfds[1], "test", 5);
```

- Para cerrar el pipe se utiliza close() dos veces, uno por cada file descriptor.

IPC, Names Pipes (FIFO)

- Es un pipe al cual se le asigna un nombre, permitiendo de esta manera que dos procesos independientes puedan abrirlo y utilizarlo.
- La FIFO es una entrada especial en el File System. Para crearla se utiliza la system call *mknod().*

```
int pfds;
mknod("Nombre", S_IFIFO | 0644, 0);

"Nombre":     Nombre de la FIFO.
S_IFIFO:      Macro que indica la creacion de una FIFO,
              va OReada con los permisos. (rw- r-- r--)
0:            Tercer argumento es el device number, en el
              caso de las FIFOs no interesa, cualquier
              valor.
```

- Luego de creada la FIFO, esta se abre con *open(),* se escribe con *write()* y lee con *read()* tal como si fuera un archivo.

ANEXOS

Estructuras y piezas de código

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
struct task_struct {
    /*
     * offsets of these are hardcoded elsewhere - touch with care
     */
    volatile long state;        /* -1 unrunnable, 0 runnable, >0 stopped */
    unsigned long flags;        /* per process flags, defined below */
    int sigpending;
    mm_segment_t addr_limit;    /* thread address space:
                                   0-0xBFFFFFFF for user-thead
                                   0-0xFFFFFFFF for kernel-thread
                                    */
    struct exec_domain *exec_domain;  /*Process Excecution Domain */
    volatile long need_resched;
    unsigned long ptrace;

    int lock_depth;             /* Lock depth */
/*
 * offset 32 begins here on 32-bit platforms. We keep
 * all fields in a single cacheline that are needed for
 * the goodness() loop in schedule().
 */
```

```
typedef struct {
        unsigned long seg;
} mm_segment_t;
```

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
long counter;
long nice;
unsigned long policy;
struct mm_struct *mm;
int processor;
/*
 * cpus_runnable is ~0 if the process is not running on any
 * CPU. It's (1 << cpu) if it's running on a CPU. This mask
 * is updated under the runqueue lock.
 *
 * To determine whether a process might run on a CPU, this
 * mask is AND-ed with cpus_allowed.
 */
unsigned long cpus_runnable, cpus_allowed;
/*
 * (only the 'next' pointer fits into the cacheline, but
 * that's just fine.)
 */
struct list_head run_list;
unsigned long sleep_time;
```

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
    struct task_struct *next_task, *prev_task;
    struct mm_struct *active_mm;
    struct list_head local_pages;
    unsigned int allocation_order, nr_local_pages;

/* task state */
    struct linux_binfmt *binfmt;
    int exit_code, exit_signal;
    int pdeath_signal;  /*  The signal sent when the parent dies  */
    /* ??? */
    unsigned long personality;
    int did_exec:1;
    unsigned task_dumpable:1;
    pid_t pid;
    pid_t pgrp;
    pid_t tty_old_pgrp;
    pid_t session;
    pid_t tgid;
    /* boolean value for session group leader */
    int leader;
```

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
/*
     * pointers to (original) parent process, youngest child, younger sibling,
     * older sibling, respectively.  (p->father can be replaced with
     * p->p_pptr->pid)
     */
    struct task_struct *p_opptr, *p_pptr, *p_cptr, *p_ysptr, *p_osptr;
    struct list_head thread_group;

    /* PID hash table linkage. */
    struct task_struct *pidhash_next;
    struct task_struct **pidhash_pprev;

    wait_queue_head_t wait_chldexit;        /* for wait4() */
    struct completion *vfork_done;          /* for vfork() */
    unsigned long rt_priority;
    unsigned long it_real_value, it_prof_value, it_virt_value;
    unsigned long it_real_incr, it_prof_incr, it_virt_incr;
    struct timer_list real_timer;
    struct tms times;
    unsigned long start_time;
    long per_cpu_utime[NR_CPUS], per_cpu_stime[NR_CPUS];
```

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
/* mm fault and swap info: this can arguably be seen as either mm-specific or thread-specific */
        unsigned long min_flt, maj_flt, nswap, cmin_flt, cmaj_flt, cnswap;
        int swappable:1;
/* process credentials */
        uid_t uid,euid,suid,fsuid;
        gid_t gid,egid,sgid,fsgid;
        int ngroups;
        gid_t   groups[NGROUPS];
        kernel_cap_t   cap_effective, cap_inheritable, cap_permitted;
        int keep_capabilities:1;
        struct user_struct *user;
/* limits */
        struct rlimit rlim[RLIM_NLIMITS];
        unsigned short used_math;
        char comm[16];
/* file system info */
        int link_count, total_link_count;
        struct tty_struct *tty; ________/* NULL if no tty */
        unsigned int locks;             /* How many file locks are being held */
/* ipc stuff */
        struct sem_undo *semundo;
        struct sem_queue *semsleeping;
```

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
/* CPU-specific state of this task */
      struct thread_struct thread;
/* filesystem information */
      struct fs_struct *fs;
/* open file information */
      struct files_struct *files;
/* namespace */
      struct namespace *namespace;
/* signal handlers */
      spinlock_t sigmask_lock;      /* Protects signal and blocked */
      struct signal_struct *sig;

      sigset_t blocked;
      struct sigpending pending;

      unsigned long sas_ss_sp;
      size_t sas_ss_size;
      int (*notifier)(void *priv);
      void *notifier_data;
      sigset_t *notifier_mask;
```

task_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
/* Thread group tracking */
      u32 parent_exec_id;
      u32 self_exec_id;
/* Protection of (de-)allocation: mm, files, fs, tty */
      spinlock_t alloc_lock;

/* journalling filesystem info */
      void *journal_info;
};
```

mm_segment_t

(/usr/src/linux-**[version]**/includes/asm-i386/processor.h)

```
typedef struct {
        unsigned long seg;
} mm_segment_t;
```

exec_domain

(/usr/src/linux-**[version]**/includes/linux/personality.h)

```
struct exec_domain {
    const char              *name;          /* name of the execdomain */
    handler_t               handler;        /* handler for syscalls */
    unsigned char           pers_low;       /* lowest personality */
    unsigned char           pers_high;      /* highest personality */
    unsigned long           *signal_map;    /* signal mapping */
    unsigned long           *signal_invmap; /* reverse signal mapping */
    struct map_segment      *err_map;       /* error mapping */
    struct map_segment      *socktype_map;  /* socket type mapping */
    struct map_segment      *sockopt_map;   /* socket option mapping */
    struct map_segment      *af_map;        /* address family mapping */
    struct module *module;                  /* module context of the ed. */
    struct exec_domain      *next;          /* linked list (internal) */
};
```

mm_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
struct mm_struct {
    struct vm_area_struct * mmap;           /* list of VMAs */
    rb_root_t mm_rb;
    struct vm_area_struct * mmap_cache;     /* last find_vma result */
    pgd_t * pgd;
    atomic_t mm_users;          /* How many users with user space? */
    atomic_t mm_count;          /* How many references to "struct mm_struct" (users count as 1) */
    int map_count;              /* number of VMAs */
    struct rw_semaphore mmap_sem;
    spinlock_t page_table_lock;             /* Protects task page tables and mm->rss */

    struct list_head mmlist;        /* List of all active mm's.  These are globally strung  together off
                                    *init_mm.mmlist, and are protected by mmlist_lock */
    unsigned long start_code, end_code, start_data, end_data;
    unsigned long start_brk, brk, start_stack;
    unsigned long arg_start, arg_end, env_start, env_end;
    unsigned long rss, total_vm, locked_vm;
    unsigned long def_flags;
    unsigned long cpu_vm_mask;
    unsigned long swap_address;
    unsigned dumpable:1;
    mm_context_t context;                   /* Architecture-specific MM context */
};
```

list_head

(/usr/src/linux-**[version]**/includes/linux/list.h)

```
struct list_head {
    struct list_head *next, *prev;
};
```

linux_binfmt

(/usr/src/linux-**[version]**/includes/linux/binfmts.h)

```
/*
 * This structure defines the functions that are used to load the binary formats that
 * linux accepts.
 */
struct linux_binfmt {
      struct linux_binfmt * next;
      struct module *module;
      int (*load_binary)(struct linux_binprm *, struct  pt_regs * regs);
      int (*load_shlib)(struct file *);
      int (*core_dump)(long signr, struct pt_regs * regs, struct file * file);
      unsigned long min_coredump;       /* minimal dump size */
};
```

completion

(/usr/src/linux-**[version]**/includes/linux/completion.h)

```
struct completion {
    unsigned int done;
    wait_queue_head_t wait;
};
```

timer_list

(/usr/src/linux-**[version]**/includes/linux/timer.h)

```
/*
 * In Linux 2.4, static timers have been removed from the kernel.
 * Timers may be dynamically created and destroyed, and should be initialized
 * by a call to init_timer() upon creation.
 *
 * The "data" field enables use of a common timeout function for several
 * timeouts. You can use this field to distinguish between the different
 * invocations.
 */
struct timer_list {
    struct list_head list;
    unsigned long expires;
    unsigned long data;
    void (*function)(unsigned long);
};
```

tms

(/usr/src/linux-**[version]**/includes/linux/times.h)

```
struct tms {
    clock_t tms_utime;
    clock_t tms_stime;
    clock_t tms_cutime;
    clock_t tms_cstime;
};
```

user_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
/*
 * Some day this will be a full-fledged user tracking system..
 */
struct user_struct {
    atomic_t __count;       /* reference count */
    atomic_t processes;     /* How many processes does this user have? */
    atomic_t files;         /* How many open files does this user have? */

    /* Hash table maintenance information */
    struct user_struct *next, **pprev;
    uid_t uid;
};
```

rlimit

(/usr/src/linux-**[version]**/includes/linux/resources.h)

```
struct rlimit {
    unsigned long   rlim_cur;
    unsigned long   rlim_max;
};
```

tty_struct

(/usr/src/linux-**[version]**/includes/linux/tty.h)

```
struct tty_struct {
        int       magic;
        struct tty_driver driver;
        struct tty_ldisc ldisc;
        struct termios *termios, *termios_locked;
        int pgrp, session;
        kdev_t device;
        unsigned long flags;
        int count;
        struct winsize winsize;
        unsigned char stopped:1, hw_stopped:1, flow_stopped:1, packet:1;
        unsigned char low_latency:1, warned:1;
        unsigned char ctrl_status;
        struct tty_struct *link;
        struct fasync_struct *fasync;
        struct tty_flip_buffer flip;
        int max_flip_cnt;
        int alt_speed;                  /* For magic substitution of 38400 bps */
        wait_queue_head_t write_wait;
        wait_queue_head_t read_wait;
        struct tq_struct tq_hangup;
        void *disc_data;
```

tty_struct

(/usr/src/linux-**[version]**/includes/linux/tty.h)

```
        void *driver_data;
        struct list_head tty_files;
#define N_TTY_BUF_SIZE 4096
/* The following is data for the N_TTY line discipline.  For historical reasons, this is included in the
        tty structure./
        unsigned int column;
        unsigned char lnext:1, erasing:1, raw:1, real_raw:1, icanon:1;
        unsigned char closing:1;
        unsigned short minimum_to_wake;
        unsigned long overrun_time;
        int num_overrun;
        unsigned long process_char_map[256/(8*sizeof(unsigned long))];
        char *read_buf;
        int read_head, read_tail, read_cnt;
        unsigned long read_flags[N_TTY_BUF_SIZE/(8*sizeof(unsigned long))];
        int canon_data;
        unsigned long canon_head;
        unsigned int canon_column;
        struct semaphore atomic_read
        struct semaphore atomic_write;
        spinlock_t read_lock;
        struct tq_struct SAK_tq; /* If the tty has a pending do_SAK, queue it here - akpm */
};
```

thread_struct

(/usr/src/linux-**[version]**/includes/asm-i386/processor.h)

```
struct thread_struct {
      unsigned long      esp0;
      unsigned long      eip;
      unsigned long      esp;
      unsigned long      fs;
      unsigned long      gs;
/* Hardware debugging registers */
      unsigned long      debugreg[8];  /* %%db0-7 debug registers */
/* fault info */
      unsigned long      cr2, trap_no, error_code;
/* floating point info */
      union i387_union i387;
/* virtual 86 mode info */
      struct vm86_struct              * vm86_info;
      unsigned long                   screen_bitmap;
      unsigned long                   v86flags, v86mask, saved_esp0;
/* IO permissions */
      int                  ioperm;
      unsigned long      io_bitmap[IO_BITMAP_SIZE+1];
};
```

fs_struct

(/usr/src/linux-**[version]**/includes/linux/processor.h)

```
struct fs_struct {
    atomic_t count;
    rwlock_t lock;
    int umask;
    struct dentry * root, * pwd, * altroot;
    struct vfsmount * rootmnt, * pwdmnt, * altrootmnt;
};
```

files_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
/*
 * Open file table structure
 */
struct files_struct {
     atomic_t count;
     rwlock_t file_lock;          /* Protects all the below members.  Nests inside tsk->alloc_lock */
     int max_fds;
     int max_fdset;
     int next_fd;
     struct file ** fd;           /* current fd array */
     fd_set *close_on_exec;
     fd_set *open_fds;
     fd_set close_on_exec_init;
     fd_set open_fds_init;
     struct file * fd_array[NR_OPEN_DEFAULT];
};
```

namespace

(/usr/src/linux-**[version]**/includes/linux/namespace.h)

```
struct namespace {
    atomic_t                count;
    struct vfsmount *       root;
    struct list_head        list;
    struct rw_semaphore     sem;
};
```

signal_struct

(/usr/src/linux-**[version]**/includes/linux/sched.h)

```
struct signal_struct {
    atomic_t                count;
    struct k_sigaction      action[_NSIG];
    spinlock_t              siglock;
};
```

sigpending

(/usr/src/linux-**[version]**/includes/linux/signal.h)

```
struct sigpending {
    struct sigqueue *head, **tail;
    sigset_t signal;
};
```